



DecimalVariable

🕒 Created time	July 4, 2026 4:58 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 4, 2026 11:50 AM

```
//Doubles - 4.12, 32, 1.234
//Decimal - 4.12, 32, 1.234

//Int32 - 00000000 00000000 00000000 00000001

decimal bankAccountBalance;

bankAccountBalance = 2.34M;

Console.WriteLine(bankAccountBalance);
```

אנו שואלים מדוע יש לנו שני סוגים שונים של משתנים שמחזיקים מספרים עשרוניים, ומדוע **Double** הוא המועדף למרות שגם הוא מאחסן מספרים עשרוניים, בשעה שיש לנו את הסוג **Decimal**.

האם **Decimal** אינו השם המושלם למספר שמכיל נקודה עשרונית

אז כן, יש לנו שני סוגים שונים, אבל יש הבדל: **Decimal** הוא המדויק מבין השניים. הוא מאחסן מספרים עד לרמת עיגול של כ-28 מקומות, כך שיכולה להיות לנו נקודה עשרונית ו-28 מספרים שלא מעוגלים עד לאחרים.

זו רמת הדיוק ש-**Decimal** יכול לאחסן.

אז למה אנחנו לא משתמשים בו להכל

בדיוק כמו בכל דבר אחר בפיתוח תוכנה, מדובר בניתוח עלות תועלת. יש עלות נוספת לשטח האחסון הנוסף שמשמש עבור כל משתנה בודד. זכרו, משתנים צריכים להיות מאוחסנים בזיכרון בדיוק כמו כל דבר אחר בקוד שלנו. לכן, כאשר אנו שומרים מקום בזיכרון עבור מספר מסוג **Decimal**, אנו צריכים לשמור מקום גדול הרבה יותר. **לדוגמה:** יש לנו **Int32**.

בזיכרון, זה מה שמאוחסן: 000000000 000000000 000000000 000000000.

גם אם המספר שלנו הוא אחד, זה מה שמאוחסן: 000000000 000000000 000000000 000000001.

אנחנו מקצים את כל המקום הזה בשביל הספרה 1.

לא ידענו שזה הולך להיות 1, ולא ידענו שזה כל מה שזה יהיה, כי אם היינו יודעים, יכולנו לאחסן אותו במקום הרבה יותר קטן בזיכרון.

אותו הדבר נכון עבור **Doubles** ו-**Decimals** בעובדה שאנו מאחסנים על סמך גודל הסוג **Decimal** דורש כמות אחסון גדולה הרבה יותר בזיכרון מאשר **Double**.

ברוב המקרים לא באמת אכפת לנו, לא באמת אכפת לנו לקבל את רמת הדיוק הקיצונית הזו.

זו הסיבה שעבור רוב הפעולות אנו משתמשים ב-**Doubles**, כי לא רק שהם זולים יותר לאחסון, אלא גם כשאנו מבצעים פעולות כמו כפל או חיבור, זול יותר לעבוד איתם ולחשב את התוצאה כשמדובר במספר קצר יותר.

לכן אנו משתמשים ב-**Doubles**.

אנו משתמשים ב-**Decimals** למונחים מאוד ספציפיים.

אנו משתמשים בהם לדבר אחד, וזה עבור כסף.

אבל עבור כל דבר מלבד יחידות אסטרונומיות וכסף, אנו נמנעים מ-**Decimal**.

הקצאת זיכרון (קבועה מראש) וביצועים

כאשר אנחנו כותבים קוד בסביבת .Net, המערכת מקצה זיכרון פיזי בשרת לפי סוג המשתנה שהצהרנו עליו, ולא לפי הערך בפועל שהכנסנו לתוכו.

```
Int32 - 000000000 000000000 000000000 000000001
```

בדוגמה של `Int32` כדי להמחיש שגם אם נשמור את המספר הקטן ביותר כמו 1, המערכת עדיין תנעל בלוק שלם של 32 ביטים בזיכרון (עם הרבה אפסים מיותרים).

אותו היגיון תקף למספרים עשרוניים.

משתנה מסוג **Decimal** תופס פי שניים יותר מקום בזיכרון מאשר משתנה מסוג **Double**.

בסביבות פיתוח מתקדמות שמתמודדות עם המון נתונים במקביל, הקצאת זיכרון מיותרת פוגעת בביצועים.

עלות מול תועלת

בעולם הפיתוח, כל החלטה נמדדת בעלות מול תועלת.

העלות של **Decimal** היא גודל הזיכרון שהוא תופס והעומס שהוא מייצר על המעבד.

המעבד יודע לבצע פעולות חשבוניות (חיבור, כפל, חלוקה) על מספרי **Double** בצורה מהירה ויעילה משמעותית מאשר על **Decimal**.

התועלת היחידה של **Decimal** היא דיוק קיצוני ללא שגיאות עיגול עד 28 מקומות אחרי הנקודה.

השאלה היא האם הדיוק הזה שווה את הירידה בביצועים.

הכלל המעשי בתעשייה

עבור 99 אחוז מהחשובים במערכות (כמו חישובי טמפרטורה, מרחקים, מידות באלמנטים של עיצוב, או חישובים סטטיסטיים), הדיוק של **Double** הוא מעל ומעבר למה שצריך, ולכן הוא הסטנדרט המוחלט בגלל המהירות שלו והחסכון בזיכרון.

המקום היחיד שבו שגיאות עיגול זעירות יכולות להוביל לאסון או לבעיות חוקיות הוא במערכות פיננסיות. לכן, הכלל הברור הוא שמתמשים ב-**Decimal** אך ורק כאשר עובדים עם כספים, מחירים או תשלומים. לכל שאר השימושים, כותבים תמיד **Double**.

~~~~~

```
bankAccountBalance = 2.34;  
bankAccountBalance = 2.34M;
```

**דוגמה:** הקומפיילר לא מאפשר לכתוב את הקוד **bankAccountBalance = 2.34**; מכיוון שברירת המחדל של המספר הזה היא **Double**, ואנחנו מנסים להצהיר עליו כ-**Decimal**, ולכן זה בלתי אפשרי.

אנחנו צריכים לזכור שאיננו רוצים לבצע המרות שגויות של סוגי נתונים.

ישנם מקרים שבהם לא ניתן לבצע המרה מרומזת (**Implicit Conversion**), כמו למשל כשמנסים להכניס ערך מסוג **Int** לתוך משתנה מסוג **Double** וזה עובד, אבל כשאנחנו מסתכלים על ה-**Double** הזה, אי אפשר פשוט להכניס אותו לתוך משתנה מסוג **Decimal**.

הסיבה לכך היא שאנחנו לא נחתוך שום חלק מהמספר במקרה הזה, אבל הערך הזה עבר טכנית עיגול בספרה ה-15 או ה-16 שלו.

הכל שם אפסים אז במקרה הספציפי של 2.34 זה לא משנה, אבל המבנה של **Double** תמיד יהיה פחות מדויק מזה של **Decimal**.

לכן, אם נבצע את ההמרה הזו, אנחנו עלולים להכניס ערך פחות מדויק לתוך המשתנה **bankAccountBalance** ממה שהוא מצפה לקבל.

**המשמעות היא שיש בעיה עם המרת הסוג הזה** – אנחנו בעצם נכניס נתונים פגומים לתוך ה-**Decimal** שלנו, ולכן הקומפיילר אוסר על כך לחלוטין.

אבל אנחנו עשויים להגיד "כן, אבל זה רק מספר, זה בסך הכל 2.34, בואו נכניס את זה ל-**Decimal**".

אז כדי לעשות זאת עבור קבועים (**Constants**) כאלה שנכתבים ישירות בקוד, אנחנו צריכים לציין במפורש שהקבוע הזה הוא **Decimal** על ידי הוספת האות **M** בסוף, כך: **bankAccountBalance = 2.34M**;

האות **M** נבחרה לנוחות שימוש, כקיצור למילה "**Money**" (כסף).

ה-**M** היא הסמן שמזהה שהמספר שנמצא לפניו הוא מסוג **Decimal**.

ברוב המקרים בקוד בסביבת ייצור, אנחנו לא מכניסים ערכים ממשיים ישירות לתוך משתנה ולא כותבים אותם ידנית (**Hard Coding**).

אנחנו בדרך כלל מקבלים את המידע הזה מקלט של משתמש או מושכים אותו מבסיס הנתונים.

במקרים כאלה, המידע יגיע למערכת כבר בסוג הנתונים הנכון.

אבל, אם אנחנו פשוט מקלידים את המספר בקוד עכשיו, והקומפילר צריך להבין בעצמו מה סוג הנתונים, הוא אוטומטית יניח שזה **Double**. מכיוון שזה לא מה שאנחנו מחפשים, אנחנו פשוט חייבים להוסיף את ה-**M** בסוף כדי לכוון אותו.

## איך הקומפילר של C# מפרש מספרים עשרוניים

כאשר כותבים מספר עם נקודה עשרונית ישירות בקוד, הקומפילר מתייחס אליו באופן אוטומטי כאל משתנה מסוג **Double**. משתנה **Double** מבוסס על מתמטיקה של נקודה צפה ומיועד לחישובים מדעיים מהירים, אך הוא נוטה ליצור שגיאות עיגול מיקרוסקופיות בספרות הרחוקות מאוד.

## למה אי אפשר להכניס Double לתוך Decimal

משתנה **Decimal** פועל בצורה שונה לחלוטין ודורש הרבה יותר משאבי זיכרון. הוא תוכנן במיוחד לעבודה עם נתונים פיננסיים (כמו מערכות סליקה בבתי חולים או אפליקציות פיננסיות), והוא שומר על דיוק מוחלט ללא שגיאות עיגול. הקומפילר חוסם המרה מרומזת מ-**Double** ל-**Decimal** כי הוא מסרב לקחת מידע מסוג שמיועד להיות פחות מדויק ולהזריק אותו לתוך משתנה שדורש דיוק קיצוני. הזרקה כזו נחשבת להכנסת מידע פגום למערכת.

## תפקידה של האות M

כדי לעקוף את הנחת המוצא של הקומפילר ולהכריח אותו לייצר את המספר באופן מדויק, מוסיפים את האות **M**. פעולה זו פוקדת על הקומפילר לא להתייחס למספר כ-**Double** בשום שלב של הקימפול, אלא להקצות לו מראש את הזיכרון והדיוק הנדרשים ל-**Decimal**.

## ה- Hard Coding מול הקוד הדינמי

ביום יום, כשבונים מערכות מידע וממשקים מודרניים, לא כותבים מספרים כמו 2.34 ישירות בתוך הפונקציות. נתוני הכספים מגיעים תמיד דרך קריאות ל-API או ישירות משאילות למסד הנתונים. כאשר המידע נשלף מתוך עמודת מסוג **Decimal** במסד הנתונים, התשתית של .NET (כמו **Entity Framework למשל**) יודעת למפות אותו ישירות למשתנה **Decimal** ב-C#, ללא צורך בשום המרה מסורבלת או תוספת של אותיות. הבעיה הזו קיימת אך ורק כשמתרגלים או מקלידים נתוני בדיקה סטטיים ישירות לתוך קובץ הקוד.

~~~~~

רוב הפעולות לא עובדות (עם Decimal).

לכן, לדוגמה, חיבור, חיסור, כפל וחילוק - אנחנו בדרך כלל לא חושבים עליהן כפעולות שמחזירות Decimal, אלא ככאלו שמחזירות Double.

שוב, זה בגלל שזול יותר (מבחינת משאבי מחשב) לבצע אותן עם Doubles.

אנחנו יכולים טכנית לעשות את זה עם Decimals, אבל כל הפונקציות המתמטיות המובנות של מיקרוסופט עובדות עם Doubles ולא עם Decimals.

אם אנחנו משתמשים ב-Decimal לאחסון מספר, רוב הסיכויים שמדובר בכסף (דולרים).

וכמה פעמים כבר יוצא לנו לכפול או לחלק את יתרת הבנק שלנו? באופן כללי, אנחנו לא מבצעים פעולות מתמטיות כמו כפל, חילוק או שורש ריבועי על יתרת בנק כשמדובר בכסף.

במקום זאת, אנחנו עושים דברים כמו חיבור וחסור.

לכן אנחנו לא רואים הרבה פונקציות מתמטיות מובנות סביב Decimal, מכיון ש-Decimal נועד בעיקר להחזיק דברים כמו כסף או יחידות אסטרונומיות.

מעבר לכך, פשוט יקר מדי לעבוד איתו, ולכן אנחנו משתמשים ב-Doubles.

ההבדל המהותי בביצועים ובייעוד בין שני סוגי המשתנים ב-.Net.

משתנה מסוג Double מבוסס על תקן שהמעבד יודע לעבוד איתו בצורה טבעית ומהירה מאוד ברמת החומרה, ולכן הוא זול ויעיל במיוחד מבחינת כוח עיבוד.

מסיבה זו, כל מחלקת Math המובנית של מיקרוסופט ב-#C מתוכננת לעבוד באופן בלעדי עם Double עבור חישובים מורכבים כמו שורש, חזקה או טריגונומטריה.

לעומת זאת, Decimal הוא משתנה כבד שמתוכנן לספק רמת דיוק אבסולוטית כדי למנוע שגיאות עיגול במספרים עשרוניים.

בגלל המבנה שלו בזיכרון, כל פעולה עליו דורשת מהמעבד הרבה יותר מאמץ, ולכן הוא נחשב יקר מבחינת ביצועים. הייעוד המרכזי של Decimal במערכות תוכנה מודרניות הוא ייצוג של כסף, כמו ביתרות עובר ושב או מערכות סליקה.

בעולמות הפיננסיים, הפעולות הנפוצות ביותר הן בסך הכל הפקדות ומשיכות, כלומר פעולות בסיסיות של חיבור וחסור.

מבחינת ההיגיון העסקי, כמעט ואין סיטואציה שבה נצטרך להוציא שורש ריבועי מיתרת הבנק של לקוח.

לכן, מיקרוסופט לא טרחה להתאים את כל הפעולות המתמטיות הכבדות לסוג משתנה זה, אלא השאירו אותן ל-Doubles, בעוד ש-Decimal נשמר למקרים הספציפיים בהם רמת הדיוק של כל אגורה היא קריטית ואין צורך בחישובים מתמטיים מתקדמים.